

Subconsultas em T-SQL — Aula (versão enxuta)

Versão para condução em sala. Material completo de referência:

[subconsultas.md](#)

Pré-requisito: `SELECT`, `JOIN`, `GROUP BY/HAVING`.

1. O que é uma subconsulta

`SELECT` entre parênteses, aninhado dentro de outro comando.

```
SELECT razao_social
FROM fiscal.contribuinte
WHERE id_contribuinte IN (
    SELECT id_contribuinte_emitente
    FROM fiscal.nota_fiscal
    WHERE valor_total > 50000.00
);
```

- **Consulta externa** filtra usando o resultado da **consulta interna**.
- Sempre entre `( )`. Não aceita `ORDER BY` (exceto com `TOP`).

2. Duas classificações rápidas

Retorno	Exemplo de uso
Escalar (1 linha, 1 coluna)	após <code>=</code> , <code>></code> , em <code>SELECT/SET</code>
Tabela (N linhas/colunas)	em <code>FROM</code> , com <code>IN/EXISTS</code>
Dependência	Característica
Independente	roda uma vez, não olha a linha externa
Correlacionada	referencia coluna da consulta externa — "reavaliada por linha"

3. Escalar — cuidado com "mais de 1 linha"

```
SELECT razao_social
FROM fiscal.contribuinte
WHERE id_contribuinte = (
    SELECT TOP (1) id_contribuinte_emitente
    FROM fiscal.nota_fiscal
    ORDER BY valor_total DESC
);
```

Sem **TOP (1) + ORDER BY** determinístico, o SQL Server lança erro se vier mais de uma linha.

4. IN / NOT IN

```
-- quem emitiu nota cancelada
WHERE id_contribuinte IN (
    SELECT id_contribuinte_emitente FROM fiscal.nota_fiscal
    WHERE situacao = 'CANCELADA'
);

-- quem NUNCA emitiu nota cancelada
WHERE id_contribuinte NOT IN (
    SELECT id_contribuinte_emitente FROM fiscal.nota_fiscal
    WHERE situacao = 'CANCELADA'
);
```

⚠ Guardar para a seção 7 — a armadilha do **NOT IN** com **NULL**.

5. EXISTS / NOT EXISTS (a forma recomendada)

```
SELECT c.razao_social
FROM fiscal.contribuinte AS c
WHERE NOT EXISTS (
    SELECT 1
```

```
FROM fiscal.nota_fiscal AS n
WHERE n.id_contribuinte_emitente = c.id_contribuinte
      AND n.situacao = 'CANCELADA'
);
```

- Testa só **se existe linha** — `SELECT 1` é convenção, sem custo extra.
- É **correlacionada** (usa `c.id_contribuinte` da linha externa).
- **Imune** ao problema de `NULL` que afeta `NOT IN`.

6. Correlacionada — exemplo direto

```
SELECT
  c.razao_social,
  (SELECT MAX(n.data_ emissao)
   FROM fiscal.nota_fiscal AS n
   WHERE n.id_contribuinte_emitente = c.id_contribuinte) AS ultima_ emissao
FROM fiscal.contribuinte AS c;
```

A subconsulta não roda sozinha: depende de `c.id_contribuinte`, que muda a cada linha.

7. A armadilha do `NOT IN` com `NULL`

```
WHERE id_contribuinte NOT IN (
  SELECT id_contribuinte_emitente FROM fiscal.nota_fiscal
);
```

Se um único `NULL` aparecer em `id_contribuinte_emitente`, a consulta **não retorna nenhuma linha** — mesmo havendo contribuintes claramente de fora da lista. Motivo: `NOT IN` vira uma cadeia de `AND (valor <> x)`, e `valor <> NULL` é `UNKNOWN`.

Correção (regra prática da equipe): use `NOT EXISTS`.

```
WHERE NOT EXISTS (  
    SELECT 1 FROM fiscal.nota_fiscal AS n  
    WHERE n.id_contribuinte_emitente = c.id_contribuinte  
);
```

8. FROM (derived table) — subconsulta como tabela

```
SELECT resumo.regime_tributario, resumo.total_contribuintes  
FROM (  
    SELECT c.regime_tributario, COUNT(DISTINCT c.id_contribuinte) AS  
total_contribuintes  
    FROM fiscal.contribuinte AS c  
    JOIN fiscal.nota_fiscal AS n ON n.id_contribuinte_emitente = c.id_contribuinte  
    GROUP BY c.regime_tributario  
) AS resumo -- alias obrigatório  
WHERE resumo.total_contribuintes > 10;
```

Alias é **obrigatório**. Útil para filtrar/reagrupar um resultado já agregado.

9. APPLY — quando a subconsulta correlacionada precisa de várias linhas

```
SELECT c.razao_social, top3.chave_acesso, top3.valor_total  
FROM fiscal.contribuinte AS c  
CROSS APPLY (  
    SELECT TOP (3) n.chave_acesso, n.valor_total  
    FROM fiscal.nota_fiscal AS n  
    WHERE n.id_contribuinte_emitente = c.id_contribuinte  
    ORDER BY n.valor_total DESC  
) AS top3;
```

Sem linhas na subconsulta

CROSS APPLY descarta a linha externa (como **INNER JOIN**)

OUTER APPLY mantém, com **NULL** (como **LEFT JOIN**)

10. Regra prática: subconsulta x JOIN x CTE

- "Existe?" → `EXISTS/NOT EXISTS`.
 - Preciso trazer colunas da outra tabela → `JOIN`.
 - Vou reaproveitar o mesmo resultado intermediário várias vezes, ou preciso de recursão → CTE (`WITH`).
 - Mais de 2–3 níveis de subconsulta aninhada → decompor em CTE.
-

11. Exercícios rápidos (para praticar em aula)

1. Liste contribuintes que emitiram nota com `valor_total > 20000`, usando `IN`.
2. Reescreva o exercício 1 com `EXISTS` (correlacionada).
3. Liste contribuintes que nunca emitiram nota, usando `NOT EXISTS`.
4. Use `CROSS APPLY` para trazer as 2 notas de maior valor de cada contribuinte.
5. **Demonstração ao vivo:** monte um `NOT IN` com dado `NULL` propositalmente e mostre o resultado vazio; corrija com `NOT EXISTS`.